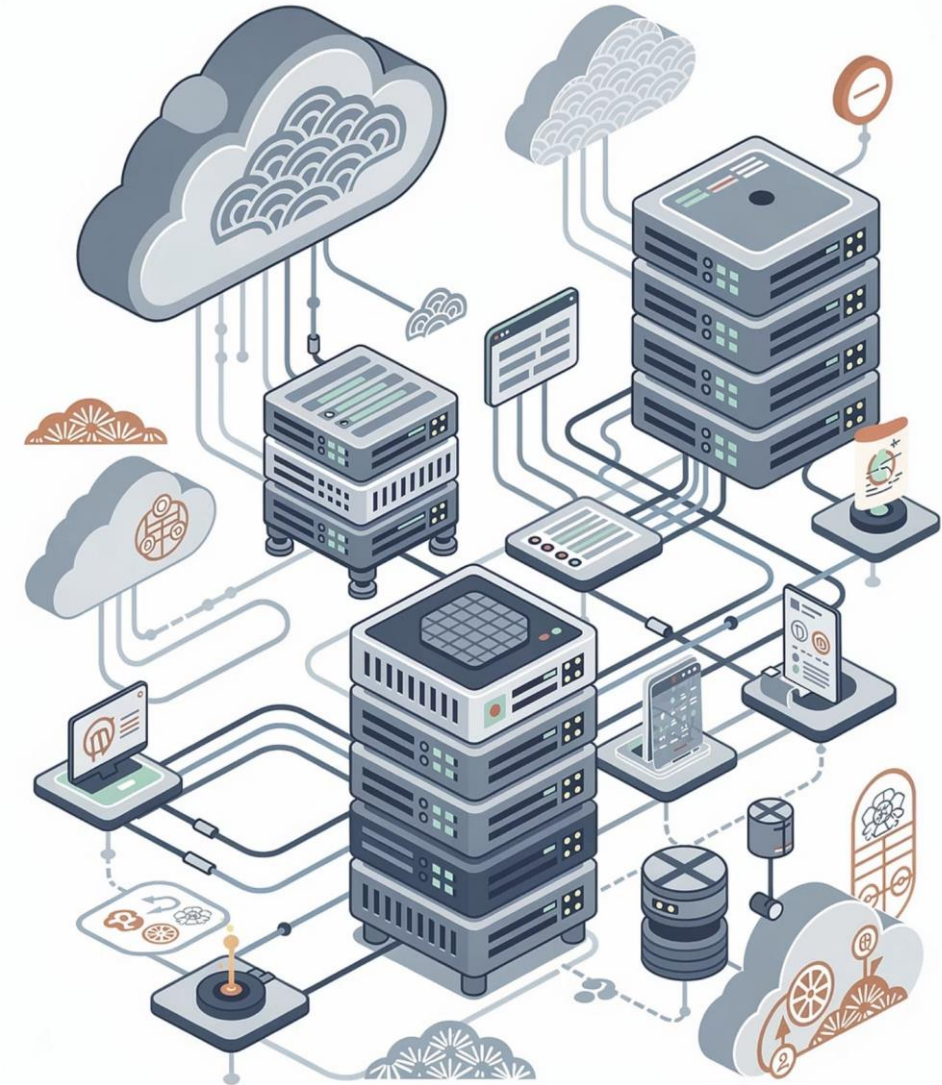




Day 6: Serverless Compute & Containers

AWS Lambda, ECS, EKS & Fargate

Today we shift beyond the traditional virtual machine. You'll master the modern cloud paradigms that eliminate idle costs, reduce operational overhead, and let you focus on what actually matters: delivering working software.

What We're Covering Today

Today's session is structured as a logical progression — from the "why" of serverless to the "how" of containers, orchestration, and compute platforms.



The Shift Beyond Servers

Why EC2 alone isn't the answer for every workload



Container Technology

VMs vs. containers, Docker, and the portability promise



AWS Lambda & Event-Driven Architecture

Reactive, millisecond-billed serverless compute



Orchestration & Fargate

ECS, EKS, and serverless container execution

The Problem with Always-On Servers

Amazon EC2 is powerful – but it comes with a hidden cost: you pay for it whether you use it or not. Idle VMs running overnight and through weekends silently drain budgets while delivering zero business value.

OS Patching Burden

Every EC2 instance requires your team to apply security updates, manage kernel versions, and reboot windows – on your schedule, not AWS's.

Idle Capacity Costs

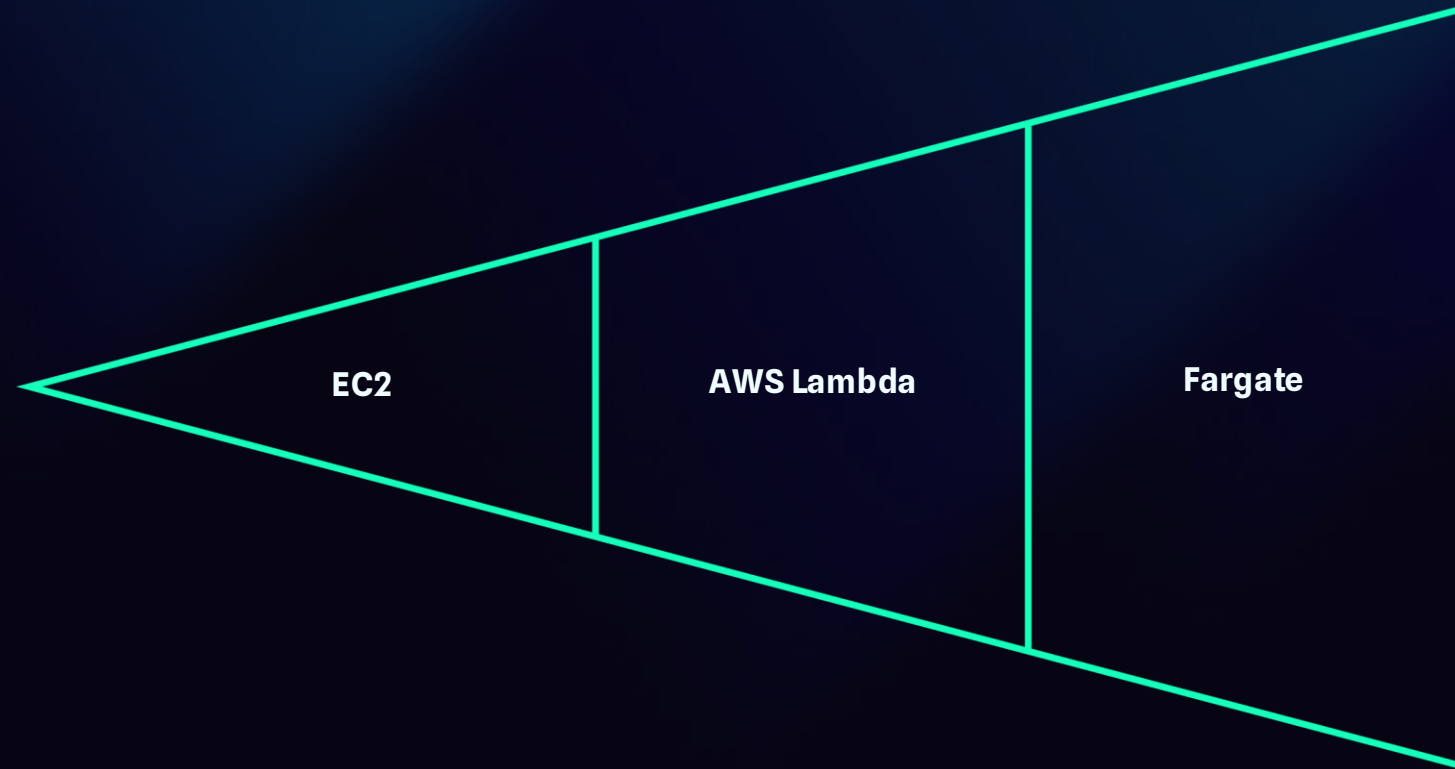
A VM running at 1% CPU at 3 AM costs exactly the same as one running at 100% CPU at noon. You pay for uptime, not utilization.

Rigid Scaling

Manual scaling rules require you to predict traffic spikes in advance. Under-provision and your app crashes. Over-provision and you waste money.

The Modern Cloud Shift

The evolution of cloud compute follows a clear trajectory: from full server management, toward platforms where AWS handles more and more of the infrastructure lifecycle.



At each stage, developers regain time previously spent on maintenance — and redirect it toward writing business logic. Your career goal is to confidently operate at every level of this stack.

Server-Based vs. Serverless: The Core Difference

These two execution models represent fundamentally different contracts between you and your cloud provider. Understanding them is foundational to every architectural recommendation you'll make in your career.

Evaluation Metric	Server-Based (Amazon EC2)	Serverless (AWS Lambda)
Billing Model	Continuous per-second/hour — pays for idle time	Request-driven per-millisecond — zero idle cost
Provisioning	Manual CPU, RAM, and AMI template selection	Code upload only; memory scales CPU proportionally
OS Maintenance	Customer applies all security patches and upgrades	AWS completely owns the underlying OS lifecycle
Scaling	Manual Auto Scaling Groups and launch configs	Automatic, concurrent, zero configuration required

"Serverless" Doesn't Mean No Servers

The Common Misconception

When students hear "serverless," they often imagine the code floating magically in the sky. In reality, AWS data centers are full of physical servers running your Lambda functions right now.

The difference: **you never see them, touch them, or pay for them when idle.**

What "Serverless" Actually Means

- AWS owns, provisions, and patches the underlying hosts
- You have zero visibility into the host OS or hardware
- Execution is ephemeral – the environment is created on demand and destroyed after the function completes
- Your responsibility starts at the function code and runtime configuration

AWS Lambda: Compute That Reacts

Lambda is the flagship AWS serverless compute service. It executes your code in isolated, ephemeral environments — but only when a specific event occurs. Think of it like a smart motion-sensor light: perfectly dark until triggered, then instantly on.

Event-Driven

Executes only in response to a defined trigger — never runs on a static loop or timer by default

Ephemeral

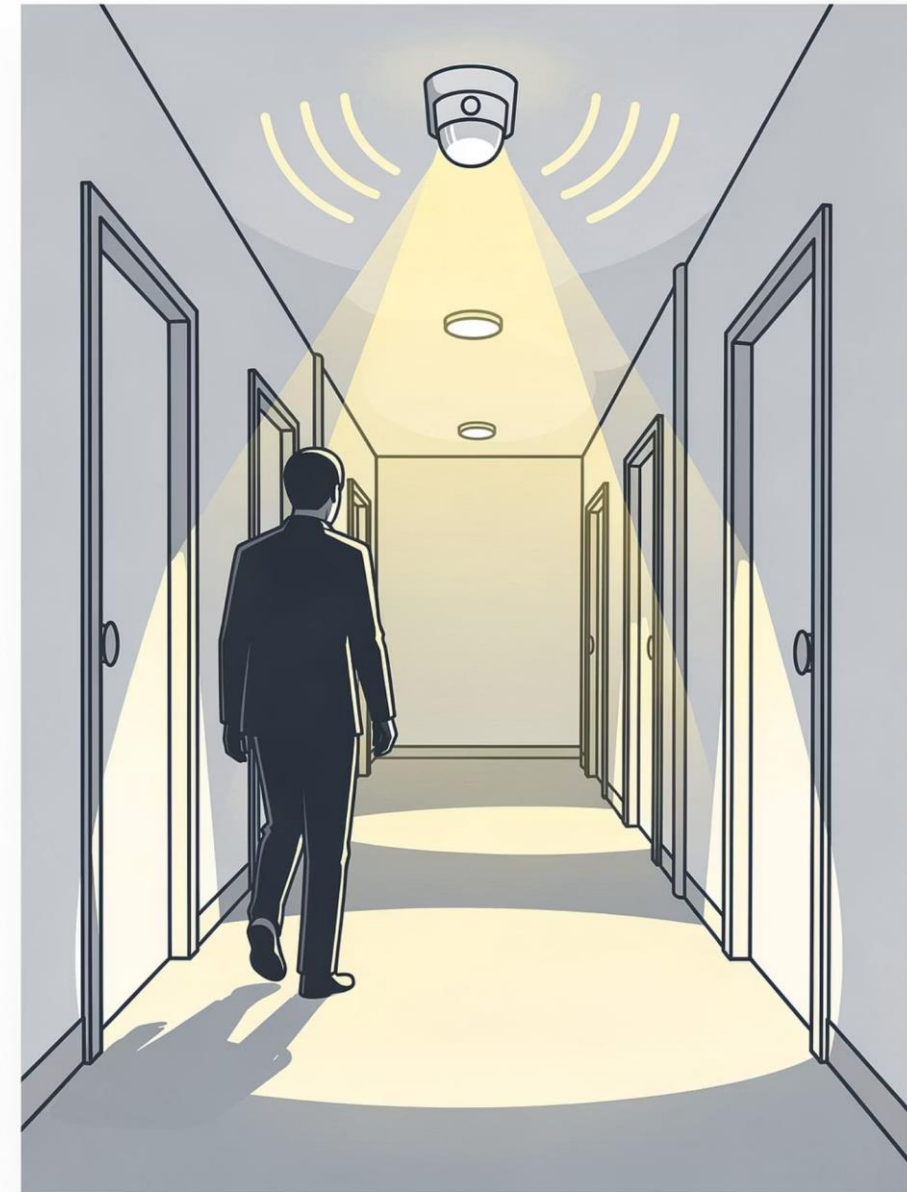
Each invocation spins up a fresh, isolated runtime container that terminates when the function completes

Millisecond Billing

You're charged only for the exact duration of active execution — rounded up to the nearest millisecond

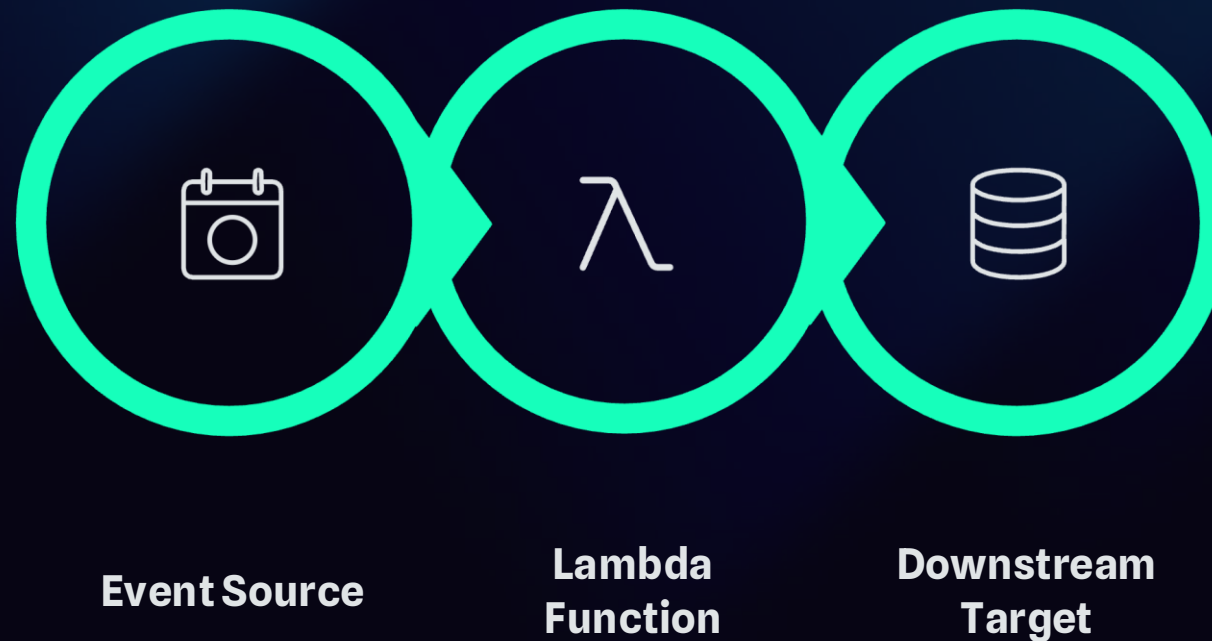
Automatic Scale

Lambda scales horizontally to thousands of concurrent executions without any configuration from you



How Lambda Works: The Event Execution Chain

Every Lambda invocation follows the same three-stage flow. Understanding this chain is critical for designing event-driven microservice architectures on AWS.



The power of this model is in its **loose coupling**: the event source doesn't know or care what Lambda does, and Lambda doesn't know what happens downstream. Each component is independently replaceable — a core microservice design principle.

Lambda: Supported Runtimes & Native Triggers

Supported Runtimes

Lambda supports a wide range of programming languages out of the box. You bring the code; AWS provides the runtime environment.

- **Node.js** – most popular for API and event processing
- **Python** – data processing, automation, ML pipelines
- **Java** – enterprise integrations and Spring Boot apps
- **C# / .NET** – Windows-native workloads migrating to cloud
- **Go & PowerShell** – systems tools and infrastructure scripts
- **Custom Runtime** – bring any language via Lambda layers

Native Event Sources

Lambda integrates directly with dozens of AWS services as event producers. No middleware required.

- **Amazon S3** – file upload, delete, or modify events
- **Amazon API Gateway** – HTTP request triggers for REST/WebSocket APIs
- **Amazon DynamoDB Streams** – react to database record changes
- **Amazon SQS** – process messages from queues asynchronously
- **Amazon EventBridge** – scheduled cron-style or rule-based triggers
- **Amazon SNS** – fan-out notification processing

Analogy: The Beverage Service Model

Let's ground these billing architectures in something tangible. Imagine you're managing beverage service for a large convention center — and you're evaluating two radically different supplier contracts.



The Commercial Brewer Rental (EC2)

You lease ten industrial coffee brewers. They run 24/7 — keeping water hot whether anyone is in the building or not. You pay a fixed hourly lease fee regardless of usage, and your facilities team handles descaling, filter changes, and safety certifications.



The On-Demand Pod Station (Lambda)

You replace the brewers with automated pod dispensers. Zero upfront cost, zero idle charges. When an attendee presses the button — an *event trigger* — exactly one cup brews, bills, and shuts off. The vendor owns and maintains the entire machine mechanism.

Beverage Model → Cloud Concept Mapping

EC2 = Commercial Brewer

You rent the entire compute instance. Billing runs 24/7 regardless of whether a single user request was processed. Your team maintains the Guest OS just like the facilities crew maintains the machine hardware.

Lambda = Pod Station

Execution is request-driven and ephemeral. The environment spins up from zero when an event fires, runs for milliseconds, and terminates. Idle cost is exactly **\$0.00**. AWS handles 100% of infrastructure maintenance.

Event Trigger = Brew Button

Compute does not run on a static loop. It fires reactively — only when a defined user action or system event occurs, such as a file upload, an API call, or a database record change.

Introducing Container Technology

What happens when your application is too complex for a single Lambda function — when it has custom dependencies, long-running processes, or must be portable across different environments? That's where containers come in.

"Package once, run anywhere. If it runs on your laptop, it runs exactly the same way in the cloud."

Containers solve the most notorious problem in software delivery: environment inconsistency. They wrap your application code, runtime, libraries, and configuration into a single, portable, self-contained unit.



Virtual Machines vs. Containers: The Stack Difference

The key distinction between VMs and containers comes down to **what layer of the stack gets abstracted**. VMs abstract hardware; containers abstract the OS kernel.

Virtual Machine Stack

- Physical Hardware
- Host OS
- Hypervisor (e.g., VMware, KVM)
- **Guest OS ← heavy overhead**
- Bins / Libraries
- Application

Running 3 VMs = 3 full Guest OS instances to boot, patch, and maintain. Startup time: **minutes**. Disk footprint: **GBs**.

Container Stack

- Physical Hardware
- Host OS
- Container Engine (e.g., Docker)
- **No Guest OS – shared host kernel**
- Bins / Libraries
- Application

Running 3 containers = 1 shared OS kernel. Startup time: **seconds**. Disk footprint: **MBs**.

Why Containers Boot Faster

The Guest OS layer is the single biggest source of overhead in traditional virtualization. Removing it has a dramatic effect on startup time, resource consumption, and operational efficiency.

~5min

VM Boot Time

BIOS, bootloader, kernel init, OS services — all must start from scratch

~5sec

Container Start Time

No OS to boot — the container engine launches the app process directly on the shared kernel

~10GB

Typical VM Image

Includes full OS installation, system libraries, and application binaries

~200MB

Typical Container Image

Contains only app code and its direct dependencies — nothing more

Docker: The Container Standard

Docker is the dominant container platform and packaging standard. A **Dockerfile** is a text-based blueprint that defines exactly how to build your application image — base OS layer, dependencies, code, and startup command.

1

Dockerfile

Text blueprint declaring base image, install steps, and run command

2

Docker Build

Assembles all layers into an immutable, versioned container image

3

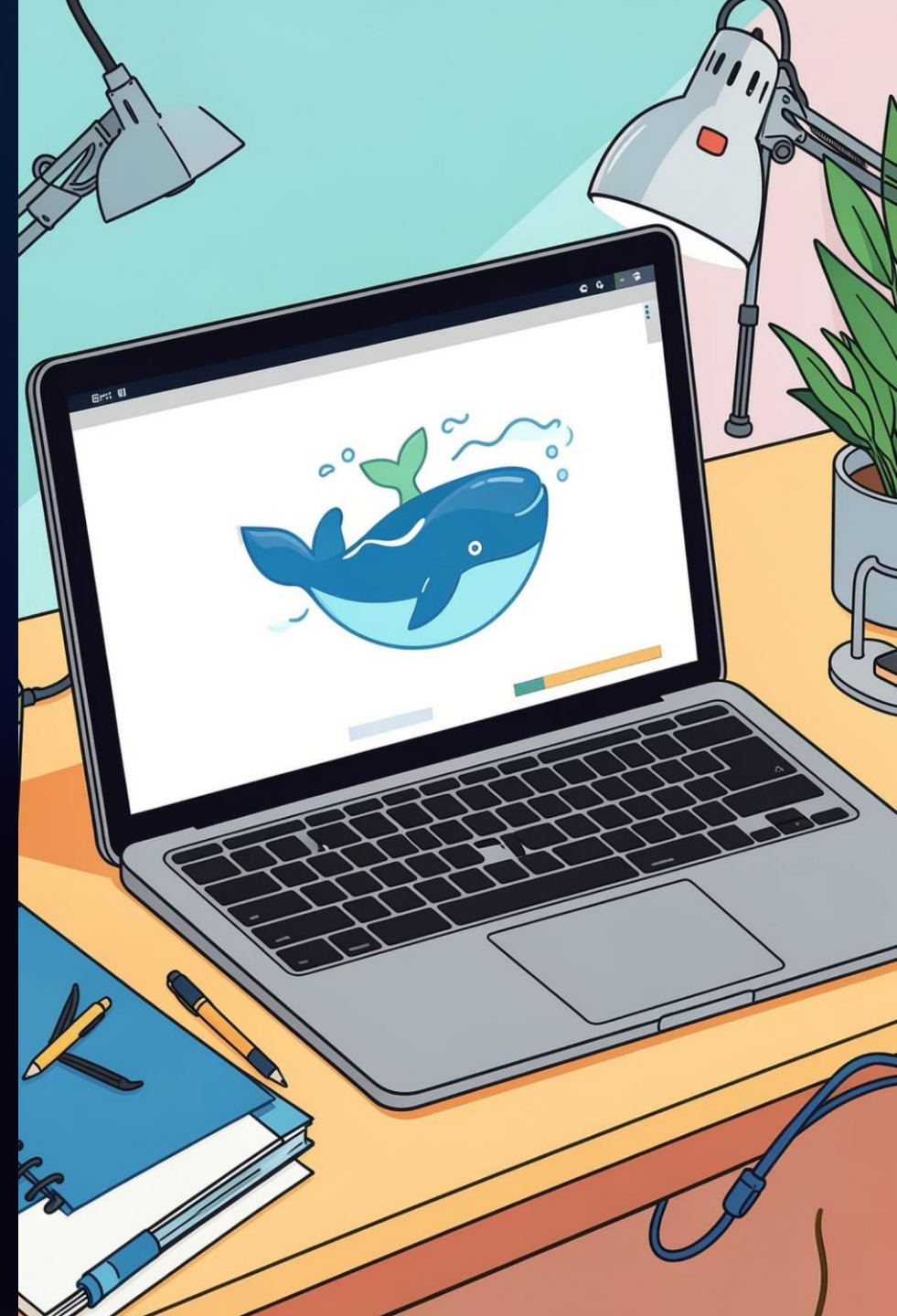
Container Registry

Image stored in Amazon ECR or Docker Hub — ready to pull anywhere

4

Container Runtime

Image runs as a live container on any host with a compatible engine



Analogy: The Standardized Shipping Container

Before standardized shipping containers, global logistics was a nightmare. Every ship, train, and truck had different hold dimensions. Every manufacturer used different packaging. Let's map this directly to software delivery.



The Old Fragmented Era (No Containers)

Every application had unique environment dependencies. Moving code from a developer laptop to a staging server meant manually reinstalling libraries, reconfiguring paths, and praying it didn't break. The classic cry: *"But it worked on my machine!"*



The Standardized Container (Docker)

Docker wraps your application and all its dependencies inside a uniform image with standard dimensions. The host environment — laptop, VM, or cloud — only needs to support the container engine. Contents are completely isolated from the outside world.

Microservices Isolation: Containers as Independent Shells

One of the most powerful properties of containers is **blast radius containment**. A failure inside one container cannot leak into a neighboring container running on the same host — just like goods locked inside independent metal shipping shells.

What This Means in Practice

- A memory leak in a payment-processing container won't crash the user-authentication container
- A bad library version in one service doesn't pollute the shared environment
- You can update, redeploy, or roll back a single container without touching other services
- Teams can own and deploy their own containers independently on different release cycles

The Microservice Promise

Each container does **one job**. One container handles PDF parsing. Another handles user authentication. Another handles email notifications.

Small, isolated, independently deployable units of functionality — this is the microservice architecture pattern.



The Orchestration Problem

Running two containers on your laptop is easy. Running two *thousand* containers across a distributed cloud cluster — across multiple Availability Zones, with health checks, rolling deployments, and auto-scaling — is an entirely different challenge.

When your container fleet grows, you need an automated harbor master: a system that schedules, monitors, scales, and routes containers without manual intervention.

This is the job of a **container orchestrator**. AWS provides two managed options: **Amazon ECS** and **Amazon EKS**. Each targets a different operational profile and organizational context.

Amazon ECS vs. Amazon EKS

Choosing between ECS and EKS is one of the most common architectural decisions you'll face as a cloud engineer. The right answer depends on your team's existing tooling and your organization's portability requirements.

Dimension	Amazon ECS	Amazon EKS
Control Plane	AWS-native, fully proprietary	Open-source Kubernetes API
Learning Curve	Low – familiar AWS JSON/YAML configs	Steeper – requires Kubernetes knowledge
AWS Integration	Deep native IAM, CloudWatch, ALB hooks	Strong but requires add-on configuration
Portability	AWS-only; not portable to other clouds	Runs on GCP, Azure, and on-premises
Best For	Fast AWS-native deployments, greenfield apps	Orgs already using Kubernetes or multi-cloud

Amazon ECS: Deep Dive

Amazon ECS is AWS's homegrown container orchestration service. It is simpler, faster to set up, and integrates more naturally with the rest of the AWS service catalog than any third-party tool.



Task Definitions

JSON blueprints that declare your container image, CPU/memory requirements, port mappings, and environment variables — the "recipe" for your container



Clusters

Logical groupings of compute capacity (EC2 instances or Fargate) where your containers are scheduled and run



Native IAM Integration

ECS tasks receive IAM roles directly, granting fine-grained AWS permissions without hardcoded credentials in your application code



CloudWatch Native

Container logs, resource utilization metrics, and alarms route seamlessly into CloudWatch with zero extra configuration

Amazon EKS: Deep Dive

Amazon EKS runs a fully managed Kubernetes control plane on your behalf. You get the full power and portability of the Kubernetes ecosystem without the burden of installing, upgrading, or operating the control plane yourself.



Managed Control Plane

AWS provisions, scales, and patches the Kubernetes API server and etcd cluster across multiple Availability Zones – you only manage worker nodes



Portability

Standard Kubernetes YAML manifests work on EKS, GKE (Google), AKS (Azure), or on-premises. No vendor lock-in at the workload layer



Rich Ecosystem

Access the full CNCF (Cloud Native Computing Foundation) ecosystem – Helm charts, Istio service mesh, Prometheus monitoring, and more



Enterprise Scale

Battle-tested at Google, Netflix, and Spotify. EKS gives large teams familiar tooling for managing thousands of containers across hybrid environments

Career Decision Guide: ECS or EKS?

When a client or employer asks which container platform to use, use these real-world signals to guide your recommendation.

Recommend ECS When...

- Team is AWS-native and has no existing Kubernetes expertise
- Speed of delivery matters more than multi-cloud portability
- The workload lives entirely within the AWS ecosystem
- Budget is constrained – ECS has lower operational overhead



Recommend EKS When...

- Engineering team already uses Kubernetes on-premises or on another cloud
- Organization requires workload portability across multiple cloud providers
- App needs advanced Kubernetes features (custom schedulers, Helm, Operators)
- The company has a long-term multi-cloud or hybrid cloud strategy

The Orchestration Analogy: Harbor Master & Rail Yard

Once hundreds of standardized containers flood into a port, a harbor master takes control. The system uses automated cranes to track IDs, load containers by weight distribution, check health status (refrigerated units have power), and automatically re-route when bottlenecks form.

Scheduling = Crane Placement

ECS/EKS decides which host node runs which container based on available CPU and memory — just like a crane placing containers optimally across rail cars

Health Checks = Inspecting Container Locks

The orchestrator continuously pings each running container. Unhealthy containers are automatically replaced — no manual intervention required

Auto-Scaling = Opening New Dock Lanes

Traffic spike? The orchestrator spins up additional container replicas to absorb load, then scales back down when demand subsides

AWS Fargate: Serverless Containers

Choosing ECS or EKS answers *how* you orchestrate containers. But you still have to decide *where* they physically run. That's the Launch Type decision — and AWS Fargate is the answer that eliminates server management entirely.

"With Fargate, you bring the container. AWS brings everything else — the server, the OS, the scaling, the patching."

Think of it like shipping your standardized cargo container to the pier edge. A fully autonomous drone fleet absorbs it, navigates it across optimal routes, and unloads it at the destination. You never lease a ship, hire a captain, or patch an engine mid-ocean.



EC2 Launch Type vs. Fargate Launch Type

When you configure a container cluster in ECS or EKS, the Launch Type determines who manages the underlying host infrastructure.

EC2 Launch Type

You manage: Provision EC2 instances, install the container agent, configure cluster capacity, apply host OS patches, manage Auto Scaling Groups, and pay for idle EC2 capacity even between container deployments.

AWS Fargate Launch Type

AWS manages: All host provisioning, OS patching, and scaling. You define vCPU and memory per task. You pay exclusively for active container runtime. No EC2 instances visible, configurable, or billable.

For most teams transitioning from VM-based ops, **Fargate is the default-right choice** — it eliminates the largest source of container operational overhead immediately.

Fargate: Granular Resource Billing

Fargate's billing model is one of its most powerful features. You declare exactly how much compute your container task needs — and pay only for what you actually use during execution.



vCPU Declaration

Choose from 0.25 to 16 vCPUs per task. Fargate provisions exactly that capacity — no over-provisioning a full instance to get the right CPU count



Per-Second Billing

Charges begin when the container task starts and end the moment it terminates. No rounding to the hour, no paying for idle base capacity between invocations



Memory Declaration

Choose from 0.5 GB to 120 GB RAM per task. CPU and memory are declared independently, giving you fine-grained control over the cost-performance tradeoff



Zero Idle Costs

When no containers are running, your compute bill is zero. Fargate's economics scale linearly with actual application usage — not infrastructure presence

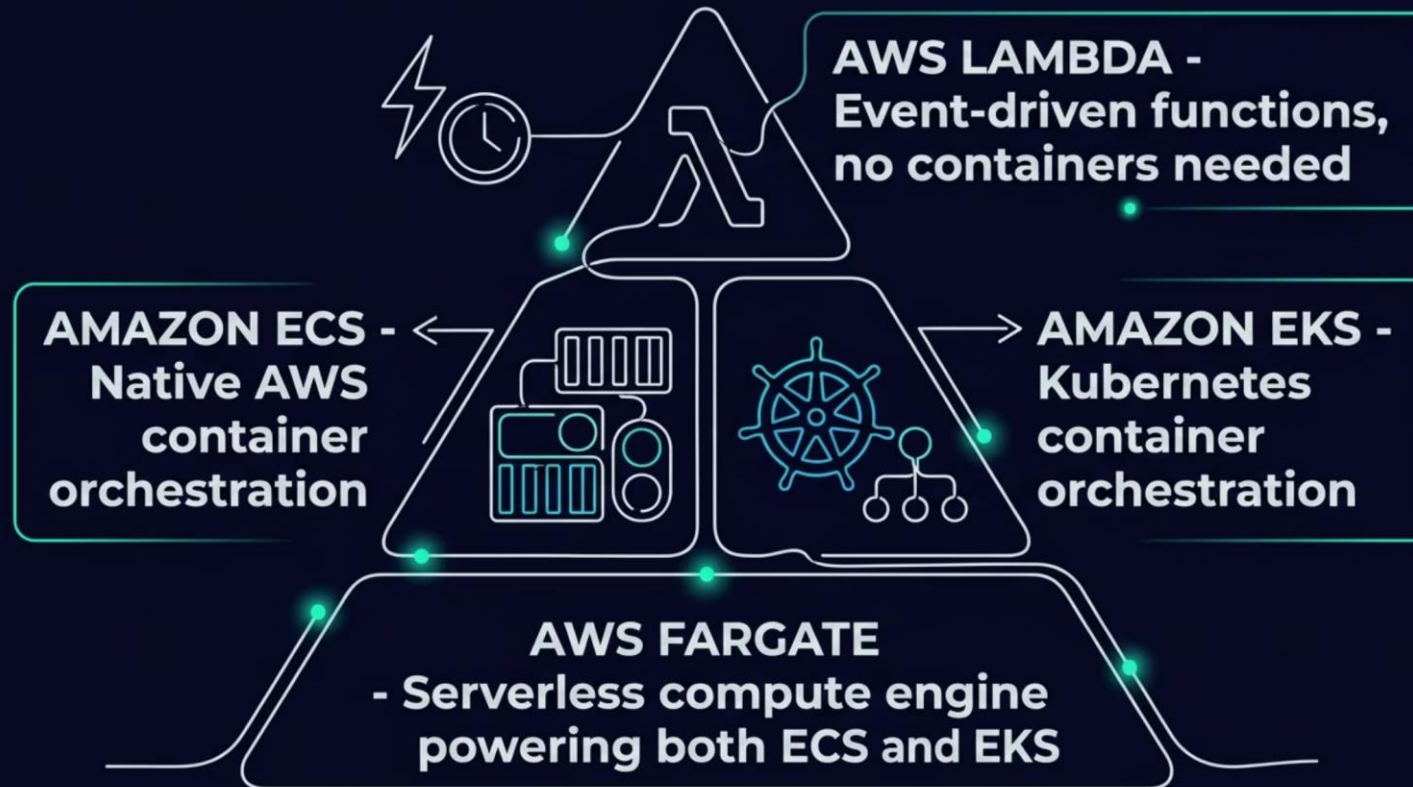
Lambda vs. Fargate: Choosing the Right Serverless Tool

Both Lambda and Fargate eliminate server management — but they serve fundamentally different workload profiles. Knowing when to use each is a core cloud architect competency.

Dimension	AWS Lambda	AWS Fargate
Workload Type	Short-lived, event-driven functions	Long-running, Dockerized applications
Max Runtime	15 minutes per invocation	No time limit — runs until stopped
Packaging	ZIP file or container image (up to 10 GB)	Docker container image (any size)
State	Stateless ephemeral execution environment	Can maintain in-process state during execution
Best Use Case	API handlers, file processing, scheduled jobs	Microservices, web apps, batch processing jobs

The Full Serverless Stack: How It Fits Together

These four AWS services aren't competing — they're complementary layers of a complete modern application architecture. Understanding how they relate is what separates a cloud support analyst from a cloud architect.



Lambda handles reactive, short-lived logic. ECS or EKS orchestrates containerized microservices. Fargate provides the serverless compute fabric underneath — eliminating host server management across both orchestrators.

CASE STUDY

SwiftConvert LLC: The Idle Monolith Trap

SwiftConvert LLC is a regional legal technology startup. They built a document processing app that converts PDF legal briefs into ADA-compliant rich text and stores outputs in Amazon S3. Their initial architecture: a single large EC2 instance running 24/7 — regardless of actual document upload activity.



CASE STUDY

The Problem: 15 Hours of Daily Idle Time

Legal filings peak between 8 AM and 5 PM, Monday through Friday. SwiftConvert's infrastructure utilization logs told a painful story.

~15hrs

Nightly Idle Window

EC2 instance running at under 1% CPU utilization every single night

<1%

Peak CPU Off-Hours

Measured CPU utilization during idle windows — yet full hourly charges accumulated

48hrs

Weekend Idle Time

Full weekend — zero document uploads, but full compute billing continued

\$0

Value Delivered Off-Hours

Zero client documents processed during those billing windows — pure waste

CASE STUDY

The Modernization: Migrating to Serverless

SwiftConvert's Lead Cloud Solutions Architect initiated a complete architectural restructuring — decomposing the monolith into an event-driven serverless pipeline.



The Lambda function runs **only when a document is actually uploaded**. At 5:01 PM when the last paralegal logs off, the compute bill drops to exactly **\$0.00**. No idle charges. No OS maintenance windows. No overnight VM babysitting.

The Financial Transformation

The results of SwiftConvert's serverless migration were immediate and measurable. AWS Lambda's millisecond billing eliminated every dollar previously wasted on idle EC2 capacity.

Month 1 Serverless Results

82%

Cost Reduction

Compute invoice dropped by over 82% in the first month of serverless operations

Infrastructure cost fell to **exactly zero** the moment the last lawyer logged off each evening. Seed capital was preserved for product development instead of idle infrastructure.

Why the Math Works

Lambda billing is calculated as:

- **Number of requests** × \$0.20 per million
- **Duration** × price per GB-second of memory allocated
- **Idle time** = \$0.00 — always

For a startup processing documents only during business hours, serverless billing perfectly mirrors actual business activity. Cost and usage become directly proportional.

CASE STUDY

The Enterprise Contract: A New Crisis Emerges

SwiftConvert's success led to a massive enterprise contract with a multinational litigation firm. On the first Monday morning, paralegals across three time zones simultaneously uploaded over **45,000 dense legal documents** within a 10-minute window.

Lambda handled horizontal scaling beautifully — thousands of concurrent function invocations fired in parallel without any configuration changes. But then the system started timing out. Not because of Lambda. Because of what Lambda was writing to.

⚠ The development team had hardcoded a write pathway to a legacy on-premises transactional database with severe I/O connection bottlenecks. Upstream elasticity overwhelmed a downstream system that couldn't scale to match.



CASE STUDY

The Downstream Blast Radius Problem

This incident reveals one of the most critical architectural lessons in event-driven design: **your entire pipeline is only as elastic as its least-scalable component.**

What Went Right

Lambda scaled horizontally to thousands of concurrent executions with zero configuration. The upstream event-handling layer performed perfectly under extreme load.

What Went Wrong

The legacy on-premises database had a fixed connection pool limit. Thousands of simultaneous Lambda write attempts saturated the pool instantly, causing timeouts and lost conversion state records.

The Fix: Buffer with a Queue

Insert an Amazon SQS queue between Lambda and the database. Lambda writes events to SQS (infinite buffer), and a separate consumer processes writes at the database's sustainable throughput rate — decoupling producer speed from consumer capacity.

Class Debate Question 1

ARCHITECTURE TRADEOFFS

Is Serverless Always Cheaper?

AWS Lambda is billed per invocation and per millisecond of execution. At low-to-moderate traffic volumes, this is dramatically cheaper than running an EC2 instance 24/7. But is it always the better choice?

When Lambda Wins

- Workloads with significant idle time (nights, weekends)
- Unpredictable or spiky traffic patterns
- Event-driven pipelines processing discrete units of work
- Applications where response time variation is acceptable

When EC2 Wins

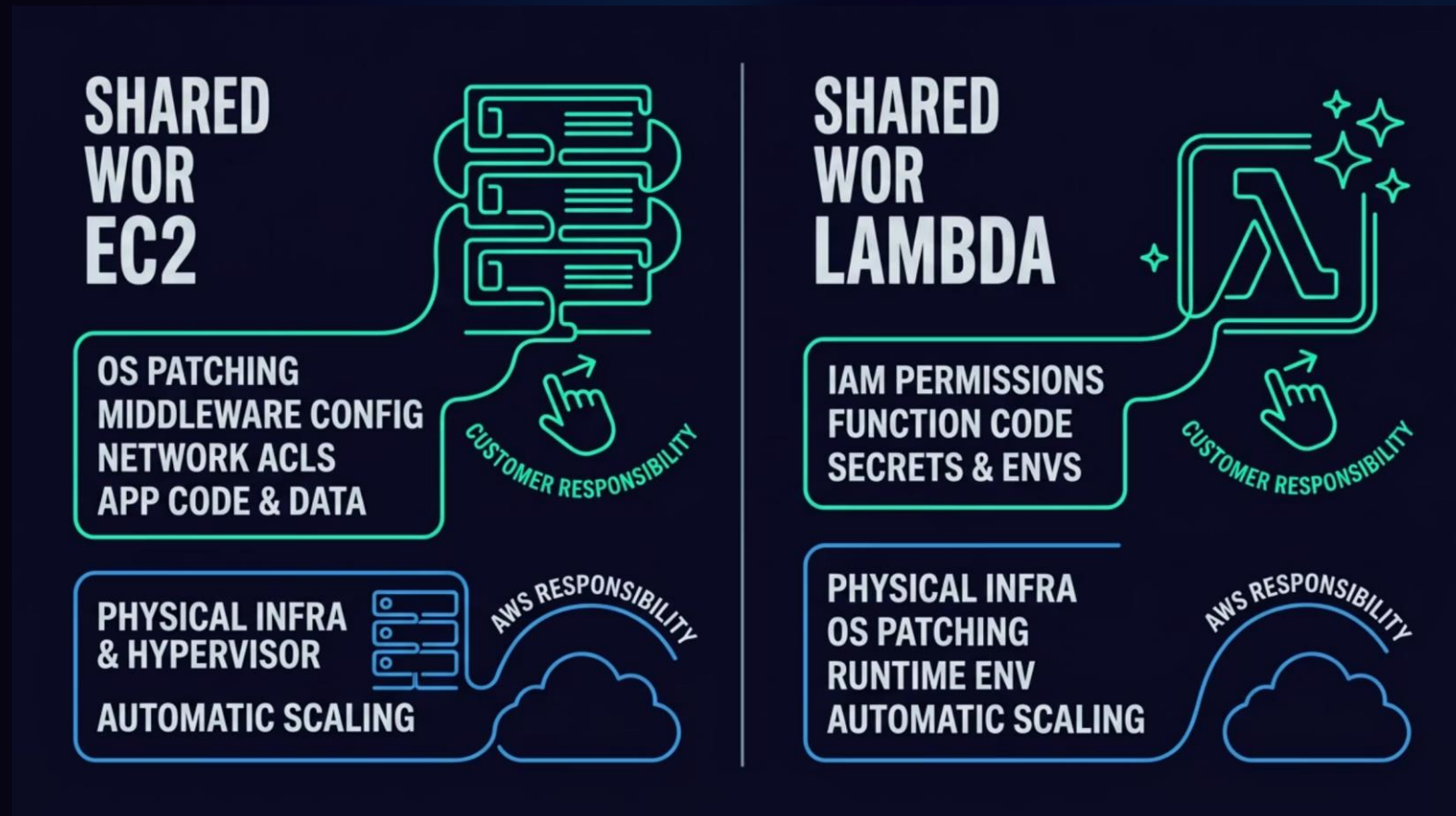
- Sustained, constant high-throughput workloads running 24/7 (EC2 Reserved Instances become cheaper)
- Long-running processes exceeding Lambda's 15-minute limit
- Applications requiring persistent in-memory state between calls
- Workloads needing specialized CPU/GPU hardware configurations

Class Debate Question 2

SHARED RESPONSIBILITY

How Did SwiftConvert's Security Obligations Change?

When SwiftConvert migrated from EC2 to Lambda, the AWS Shared Responsibility Model shifted the boundary between "AWS's job" and "your job" significantly. This is a critical concept for any compliance-conscious organization.



What SwiftConvert Handed Off to AWS

The serverless migration didn't eliminate security responsibility — it *redistributed* it. Understanding exactly where the boundary moved is essential for compliance audits and security architecture reviews.

AWS Now Owns (Lambda)

- Physical server and network hardware security
- Hypervisor and host OS patching and hardening
- Runtime environment (Node.js engine) lifecycle and updates
- Automatic execution environment isolation between customers
- DDoS protection and infrastructure-layer scaling

SwiftConvert Still Owns

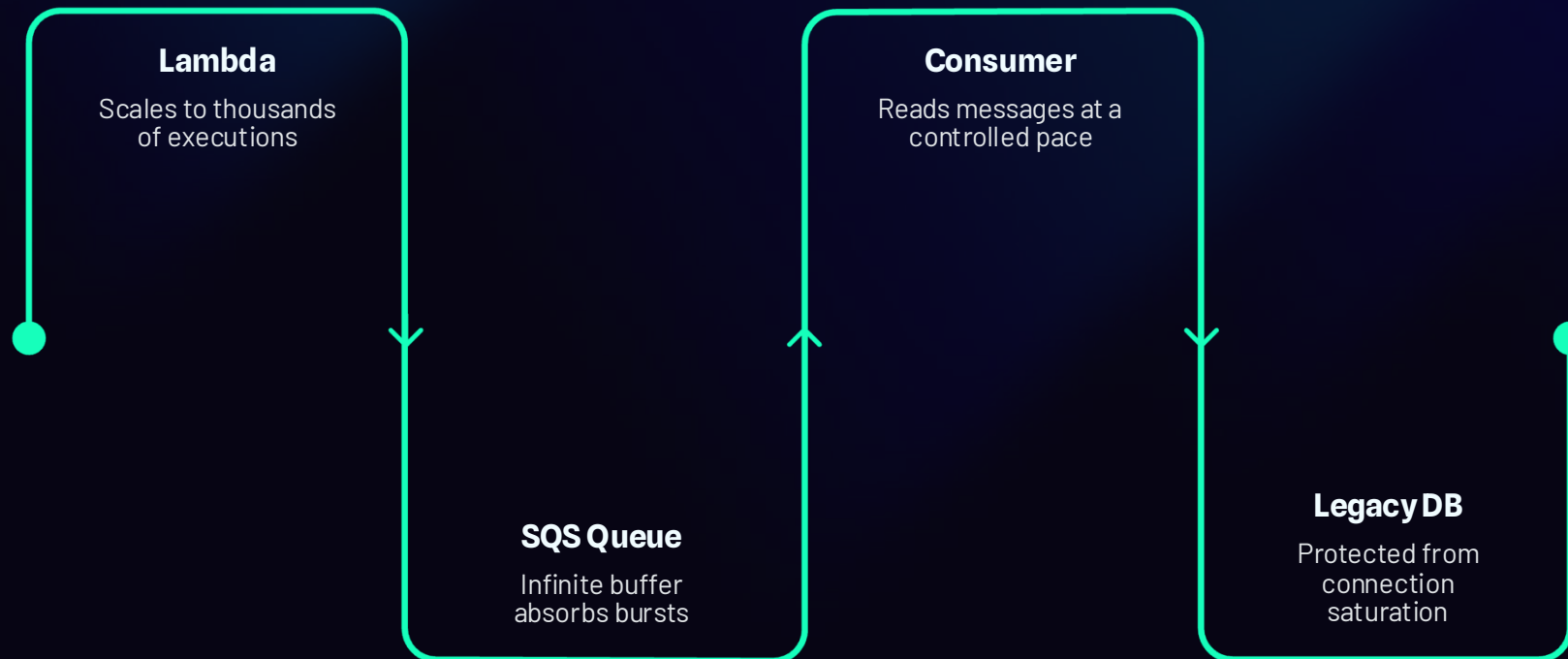
- IAM execution role permissions scoped to least privilege
- Application code quality, input validation, and injection prevention
- Secrets management (API keys, DB passwords via AWS Secrets Manager)
- Environment variable encryption at rest
- VPC configuration if Lambda needs private network access
- Data encryption in S3 and DynamoDB (at-rest and in-transit)

Class Debate Question 3

BLAST RADIUS MANAGEMENT

How Do You Protect Slow Downstream Systems from Fast Upstream Scalars?

SwiftConvert's crisis perfectly illustrates the "thundering herd" anti-pattern: upstream elastic components scale instantly, while downstream legacy systems have fixed capacity limits. This is one of the most common failure modes in serverless architectures.



The SQS queue acts as a **shock absorber** — decoupling the speed of the producer (Lambda) from the throughput capacity of the consumer (legacy DB). This is the standard AWS pattern for protecting fixed-capacity downstream systems.

Downstream Protection Patterns

Beyond SQS, cloud engineers have a toolkit of patterns for managing the blast radius when elastic upstream components meet capacity-constrained downstream systems.



Amazon SQS Buffering

Place a queue between Lambda and the database. Lambda writes at full speed; a separate consumer drains the queue at a controlled, sustainable rate. Buffer absorbs burst traffic gracefully.



Lambda Concurrency Limits

Set a reserved concurrency limit on your Lambda function to cap the maximum simultaneous invocations — preventing the function from overwhelming downstream connection pools.



Circuit Breaker Pattern

Implement logic that detects downstream failure and temporarily stops sending new requests — preventing cascading failures while giving the downstream system time to recover.



Upgrade the Downstream Layer

Replace legacy fixed-capacity databases with elastic alternatives like Amazon DynamoDB or Amazon Aurora Serverless — which scale automatically to match upstream throughput.

Key Takeaways: Day 6 Summary

Today you gained the foundational knowledge to design, evaluate, and recommend modern serverless and container-based architectures on AWS. Here's what to lock in before tomorrow's lab.

1 **Serverless means AWS manages the infrastructure — not that servers don't exist**

Lambda and Fargate both run on physical AWS hardware. Your responsibility starts at the code and configuration layer.

3 **ECS for AWS-native simplicity; EKS for Kubernetes portability**

Match the orchestration tool to your team's existing skills and your organization's multi-cloud strategy — not just what's newest.

2 **Lambda is for reactive, short-lived logic; Fargate is for long-running Dockerized workloads**

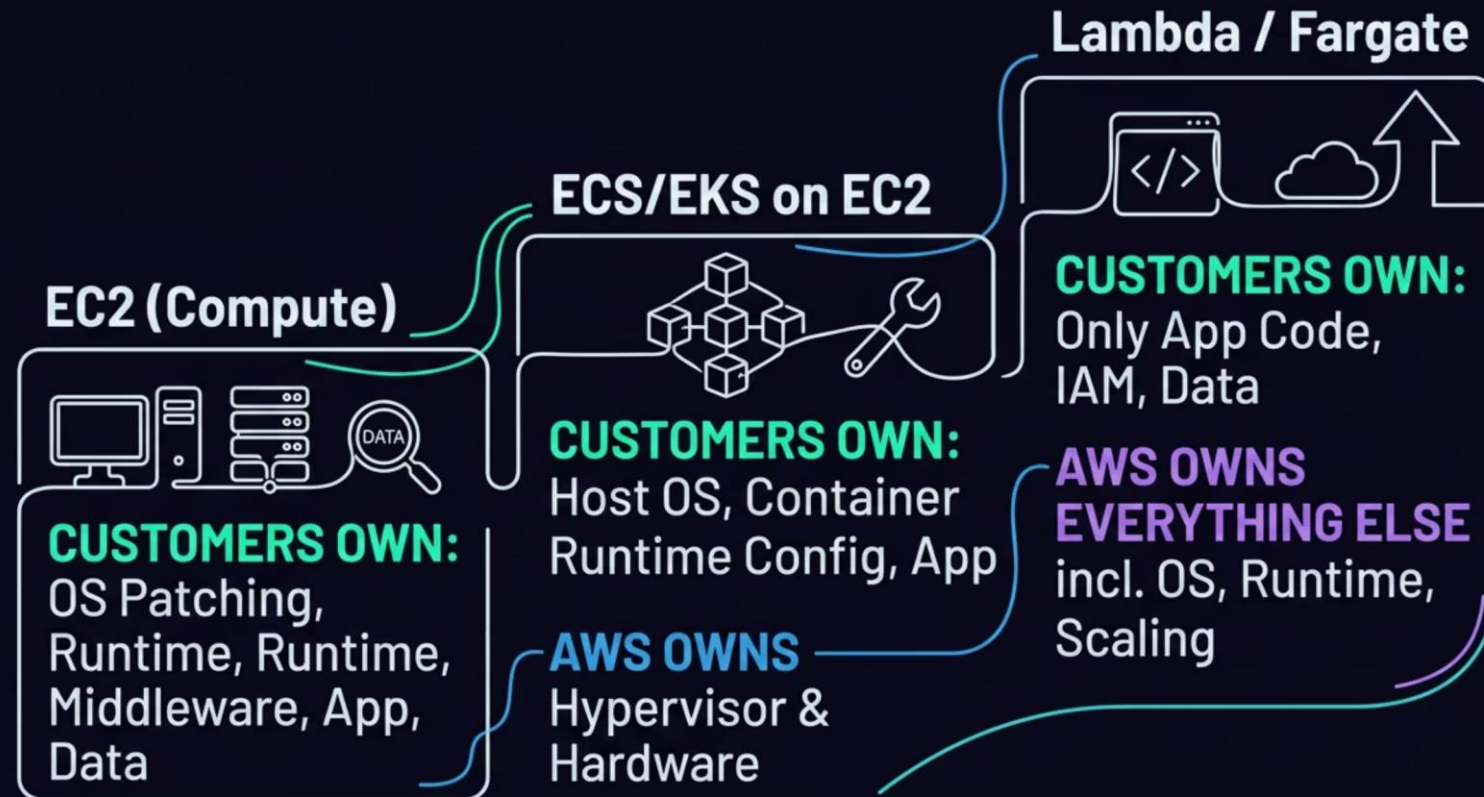
Choose based on execution duration, packaging requirements, and whether the workload is event-driven or service-oriented.

4 **Elastic upstream + fixed-capacity downstream = architectural failure mode**

Always design downstream systems (databases, queues, APIs) to match the elasticity of your upstream compute layer — or buffer between them.

The Shared Responsibility Cheat Sheet

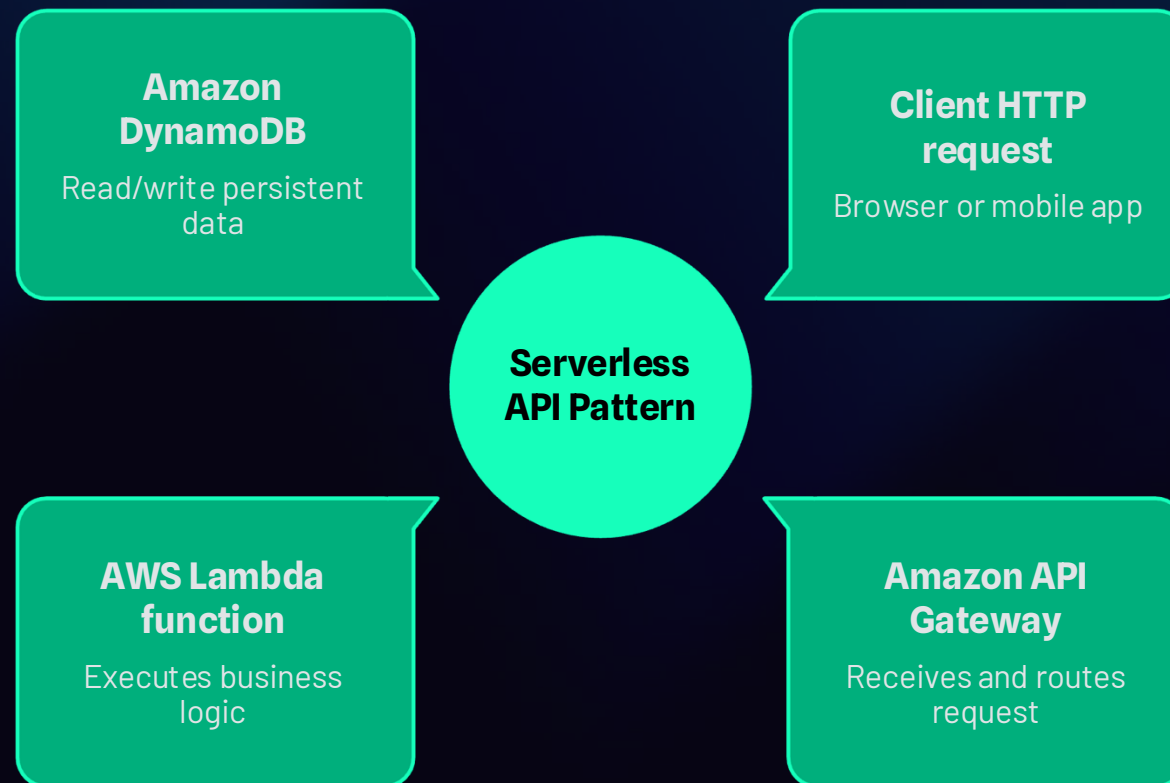
One of the most tested concepts in AWS certifications — and one of the most important in real-world security reviews. Keep this mental model sharp.



As you move up the abstraction stack — from EC2 to containers to serverless — your security surface area **shrinks**, but it never reaches zero. You always own your application code, your IAM policies, and your data.

Architecture Pattern: Serverless Web API

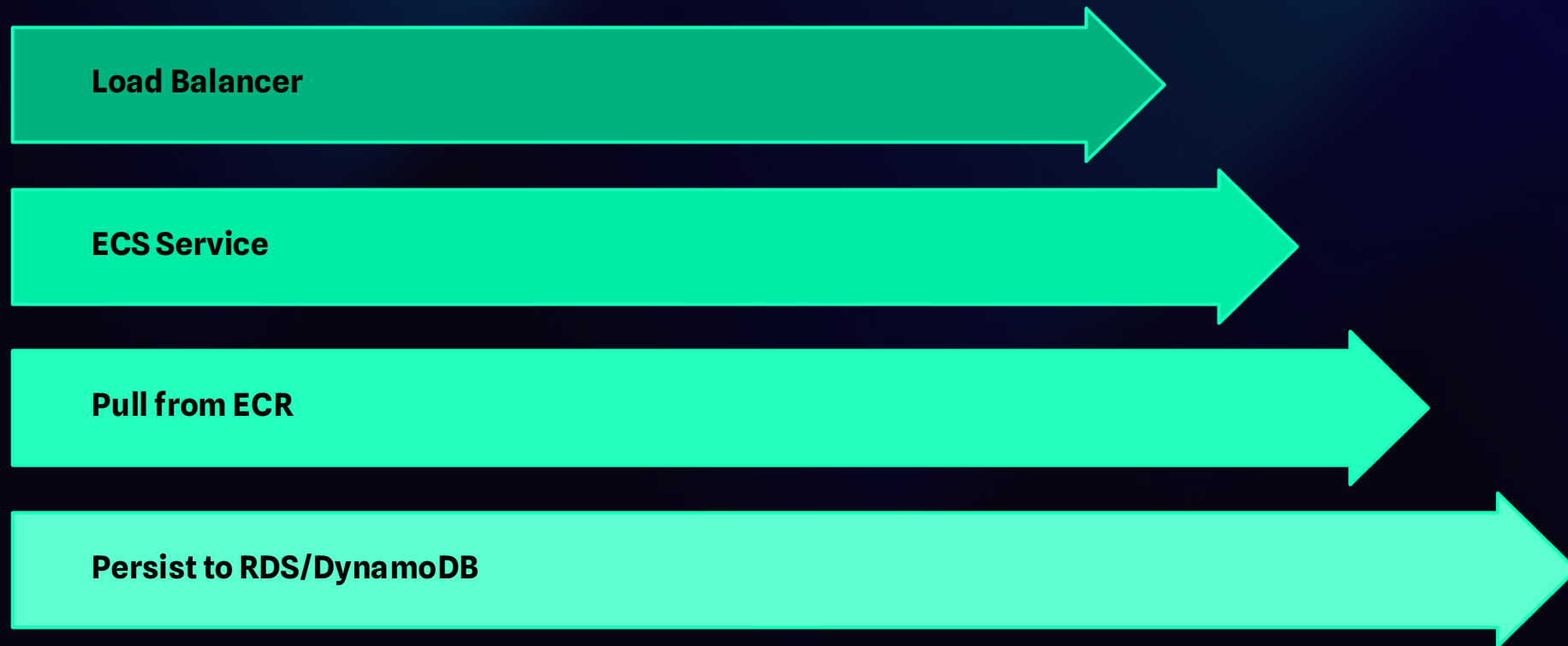
This is one of the most common serverless architecture patterns you'll build and support in your career. Every component is fully managed – no servers, no OS patches, no capacity planning.



This pattern scales from zero to millions of requests without any configuration changes. Cost is strictly proportional to actual usage. With API Gateway + Lambda + DynamoDB, you have a complete, production-grade, serverless backend – all billed per request.

Architecture Pattern: Containerized Microservice on Fargate

When your workload needs more than 15 minutes of runtime, requires complex dependencies, or runs as a persistent service — this Fargate-based pattern is your go-to design.



The Application Load Balancer distributes traffic across Fargate tasks. ECS automatically replaces unhealthy tasks and scales task count based on CPU or memory alarms. You define the Docker image and resource requirements — Fargate handles the rest of the infrastructure.

Amazon ECR: Where Your Container Images Live

Before ECS, EKS, or Fargate can run your container, they need to pull the Docker image from somewhere. Amazon Elastic Container Registry (ECR) is AWS's fully managed private container image registry.

Private & Secure

Images stored in ECR are private by default, access-controlled by IAM policies — no public Docker Hub exposure of proprietary application code

Vulnerability Scanning

ECR automatically scans images for known OS and library CVEs on push — surface security issues before they ever reach production

Lifecycle Policies

Automatically expire old image versions to control storage costs and prevent accumulation of stale, unpatched image tags in your registry

Cross-Region Replication

Replicate images to multiple AWS regions automatically — enabling low-latency container pulls for globally distributed application deployments

Cost Optimization: Choosing the Right Compute Model

Every architecture decision has a cost implication. Use this framework to select the most economical compute model based on your workload's actual runtime profile.



Lambda: best for irregular bursty traffic with idle gaps, lowest cost for infrequent workloads.



Fargate: best for containerized services with variable load, no idle infrastructure cost.



EC2 On-Demand: best for unpredictable short-term workloads, pay by the hour.



EC2 Reserved Instance: best for steady-state 24/7 workloads with predictable traffic, 40-60% savings vs On-Demand.

The Complete Analogy Recap

Before the lab, let's cement both analogies into a single reference summary. These mappings will help you explain these concepts to non-technical stakeholders — a critical skill for cloud support and solutions roles.

EC2 = Commercial Brewer on 24/7 Lease

You pay for the machine constantly. Your team maintains it. Whether it's brewing or sitting cold, the bill runs.

Lambda = On-Demand Pod Station

Zero cost at idle. Brews exactly one cup per button press. Vendor maintains the hardware. You pay per cup brewed.

Docker = Standardized Metal Shipping Container

Package your app and dependencies in a uniform box. Every host handles it identically, regardless of what's inside.

ECS/EKS = Harbor Master System

Automates container scheduling, health checking, and scaling across your fleet — like automated cranes at a container port.

Fargate = Autonomous Drone Ship Fleet

You deliver the container to the pier. AWS sails it, docks it, and unloads it. You never own or maintain a hull.

Lab Preview: What You'll Build Today

The best way to internalize these concepts is to build and observe them running in your own AWS account. Today's lab exercises follow the natural progression of the lecture.



Deploy a Lambda Function

Write a Node.js function, configure an S3 event trigger, and observe the execution logs in CloudWatch



Build and Push a Docker Image

Write a Dockerfile, build an image locally, and push it to Amazon ECR using the AWS CLI



Deploy to ECS on Fargate

Create a Task Definition, configure an ECS Service, and watch Fargate provision and run your container without any EC2 instances



Observe Cost & Metrics

Compare CloudWatch metrics and estimated billing between idle EC2, active Lambda, and running Fargate tasks

Certification Alignment: What Today Maps To

Today's content directly maps to high-weight domains in two major AWS certifications. If you're on a certification track, flag these concepts for focused review.

AWS Certified Cloud Practitioner (CLF-C02)

- Identifying serverless vs. server-based compute models
- Understanding the AWS Shared Responsibility Model boundary shifts
- Recognizing Lambda, ECS, EKS, and Fargate use cases
- Core billing models: on-demand vs. per-request

AWS Solutions Architect – Associate (SAA-C03)

- Designing event-driven architectures with Lambda + S3 + SQS
- Choosing between ECS and EKS for container orchestration scenarios
- Configuring Fargate task definitions with correct vCPU and memory
- Applying concurrency limits and queue-based buffering for blast radius control

Glossary: Day 6 Key Terms

Knowing the vocabulary is essential for exam performance, job interviews, and client-facing conversations. Review these before the next session.

Term	Definition
Serverless	A model where the cloud provider manages all underlying infrastructure; you provide code and configuration only
Event-Driven Architecture	A design pattern where compute executes reactively in response to defined system or user events
Container	A lightweight, portable package containing application code and all its dependencies, sharing the host OS kernel
Container Image	An immutable, versioned snapshot of a container's filesystem, produced by a Docker build process
Orchestration	Automated management of container deployment, scaling, health checking, and networking across a cluster
Launch Type	The compute infrastructure model used to run ECS/EKS containers: EC2 (self-managed hosts) or Fargate (serverless hosts)
Blast Radius	The scope of impact when a failure or overload condition propagates from one system component to downstream dependencies



Day 6: Complete

You've covered the full spectrum of modern AWS compute — from reactive serverless functions to orchestrated containerized microservices. These are the technologies powering the applications used by millions of people every day.

Next Up: Day 7

Storage deep dive — Amazon S3, EBS, EFS, and how they integrate with today's serverless and container architectures

Before Next Class

Complete the lab, review the three debate questions, and read the AWS Fargate pricing page to internalize the per-second billing model

Office Hours

Available Thursday 4–6 PM. Bring your lab screenshots and any architectural questions from today's SwiftConvert case study